Lab Sheet 5 — SQL Joins

1. Demonstrate an INNER JOIN between Student and Department.

```
1  SELECT s.Name, d.Dept_Name
2  FROM Student s INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid

Name	Dept_Name
Anshika	Computer Science
Radha	Computer Science
Jayshree	Computer Science
Shreya	Computer Science
Rahul	Computer Science
Priya	Computer Applications
Aman	Information Technology
Neha	Computer Science
Rohit	Computer Applications
Simran	Information Technology

2. Display student names along with their department names using INNER JOIN.

```
3  SELECT s.Student_ID, s.Name, d.Dept_Name
4  FROM Student s INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid

Student_ID	Name	Dept_Name
101	Anshika	Computer Science
102	Radha	Computer Science
103	Jayshree	Computer Science
105	Shreya	Computer Science
107	Rahul	Computer Science
108	Priya	Computer Applications
109	Aman	Information Technology
110	Neha	Computer Science
111	Rohit	Computer Applications
112	Simran	Information Technology

3. Display employee names along with their department names.

```
5  SELECT e.Emp_Name, d.Dept_Name
6  FROM Employee e INNER JOIN Department d ON e.Dept_ID = d.Dept_ID;
```

Result Grid

Emp_Name	Dept_Name
Aarav	Computer Science
Diya	Human Resources
Kabir	Information Technology
Meera	Mechanical
Rohan	Computer Science
Sneha	Information Technology
Vikram	Computer Science

4. Display students along with their course details.

```
7  SELECT s.Name, c.Course_Name, c.Duration, c.Fees
8  FROM Student s INNER JOIN Course c ON s.Course_ID = c.Course_ID;
```

Result Grid

Name	Course_Name	Duration	Fees
Anshika	B.Tech CSE	4 Years	90000.00
Radha	B.Tech CSE	4 Years	90000.00
Jayshree	B.Tech AIML	4 Years	95000.00
Ruhani	B.Tech CSE	4 Years	90000.00
Shreya	B.Tech CSE	4 Years	90000.00
Maya	B.Tech CSE	4 Years	90000.00
Rahul	B.Tech CSE	4 Years	90000.00
Priya	BCA	3 Years	60000.00
Aman	B.Tech IT	4 Years	88000.00
Neha	B.Tech CSE	4 Years	90000.00
Rohit	BCA	3 Years	60000.00
Simran	B.Tech IT	4 Years	88000.00

5. Find students belonging to a particular department using INNER JOIN.

```
SELECT s.Name, d.Dept_Name
FROM Student s INNER JOIN Department d ON s.Dept_ID = d.Dept_ID
WHERE d.Dept_ID = 1;
```

Result Grid

Name	Dept_Name
Anshika	Computer Science
Radha	Computer Science
Jayshree	Computer Science
Shreya	Computer Science
Rahul	Computer Science
Neha	Computer Science

6. Demonstrate a LEFT OUTER JOIN between Student and Department.

```
SELECT s.Name, d.Dept_Name
FROM Student s LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid

Name	Dept_Name
Anshika	Computer Science
Radha	Computer Science
Jayshree	Computer Science
Ruhani	NULL
Shreya	Computer Science
Maya	NULL
Rahul	Computer Science
Priya	Computer Applications
Aman	Information Technology
Neha	Computer Science
Rohit	Computer Applications
Simran	Information Technology

7. Display all students, including those who do not have a matching department.

```
SELECT s.Student_ID, s.Name, d.Dept_Name
FROM Student s LEFT OUTER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid

Student_ID	Name	Dept_Name
101	Anshika	Computer Science
102	Radha	Computer Science
103	Jayshree	Computer Science
104	Ruhani	NULL
105	Shreya	Computer Science
106	Maya	NULL
107	Rahul	Computer Science
108	Priya	Computer Applications
109	Aman	Information Technology
110	Neha	Computer Science
111	Rohit	Computer Applications
112	Simran	Information Technology

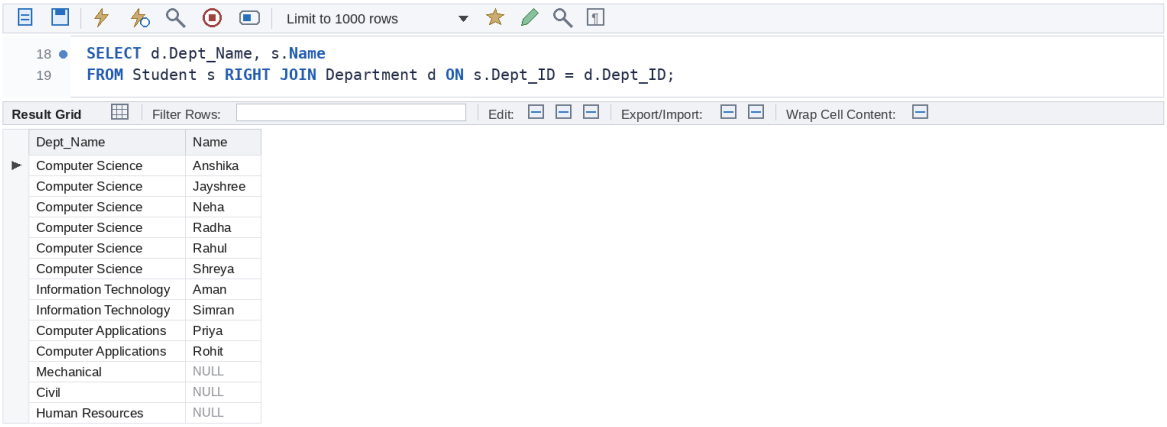
8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

```
SELECT s.Name, d.Dept_Name
FROM Student s RIGHT JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid

Name	Dept_Name
Anshika	Computer Science
Jayshree	Computer Science
Neha	Computer Science
Radha	Computer Science
Rahul	Computer Science
Shreya	Computer Science
Aman	Information Technology
Simran	Information Technology
Priya	Computer Applications
Rohit	Computer Applications
NULL	Mechanical
NULL	Civil
NULL	Human Resources

9. Display all departments, including departments having no students.

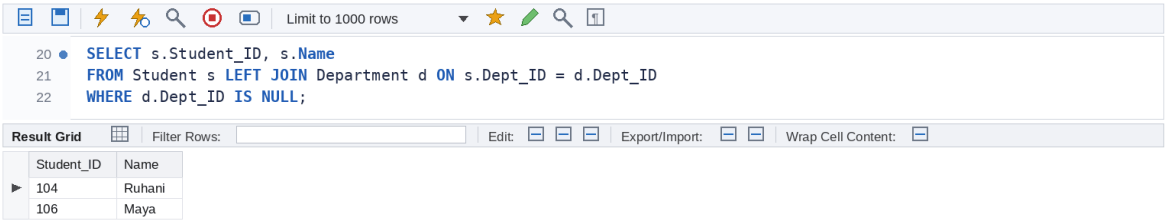


```
18  SELECT d.Dept_Name, s.Name
19  FROM Student s RIGHT JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Dept_Name	Name
Computer Science	Anshika
Computer Science	Jayshree
Computer Science	Neha
Computer Science	Radha
Computer Science	Rahul
Computer Science	Shreya
Information Technology	Aman
Information Technology	Simran
Computer Applications	Priya
Computer Applications	Rohit
Mechanical	NULL
Civil	NULL
Human Resources	NULL

10. Find students who are not assigned to any department using LEFT JOIN.

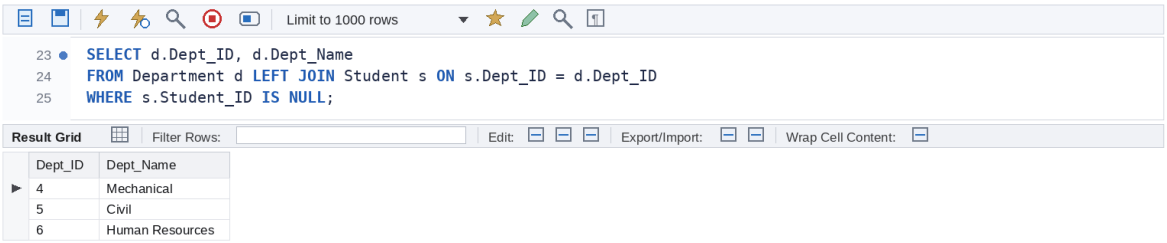


```
20  SELECT s.Student_ID, s.Name
21  FROM Student s LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
22  WHERE d.Dept_ID IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Student_ID	Name
104	Ruhani
106	Maya

11. Find departments having no students using RIGHT/LEFT JOIN.

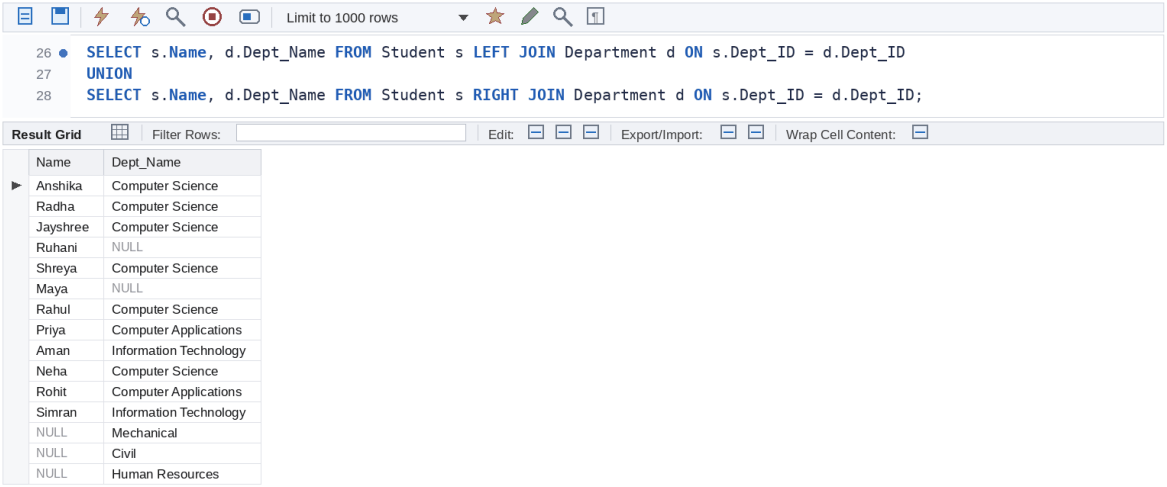


```
23  SELECT d.Dept_ID, d.Dept_Name
24  FROM Department d LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
25  WHERE s.Student_ID IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Dept_ID	Dept_Name
4	Mechanical
5	Civil
6	Human Resources

12. Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround where required.



```
26  SELECT s.Name, d.Dept_Name FROM Student s LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
27  UNION
28  SELECT s.Name, d.Dept_Name FROM Student s RIGHT JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Name	Dept_Name
Anshika	Computer Science
Radha	Computer Science
Jayshree	Computer Science
Ruhani	NULL
Shreya	Computer Science
Maya	NULL
Rahul	Computer Science
Priya	Computer Applications
Aman	Information Technology
Neha	Computer Science
Rohit	Computer Applications
Simran	Information Technology
NULL	Mechanical
NULL	Civil
NULL	Human Resources

13. Demonstrate a CROSS JOIN between Student and Course.

```
29  SELECT s.Name, c.Course_Name
30  FROM Student s CROSS JOIN Course c;
```

Result Grid

Name	Course_Name
Anshika	B.Tech CSE
Anshika	B.Tech AIML
Anshika	B.Tech IT
Anshika	BCA
Anshika	B.Tech Civil
Radha	B.Tech CSE
Radha	B.Tech AIML
Radha	B.Tech IT
Radha	BCA
Radha	B.Tech Civil
Jayshree	B.Tech CSE
Jayshree	B.Tech AIML
Jayshree	B.Tech IT
Jayshree	BCA
Jayshree	B.Tech Civil

14. Calculate the number of possible Student-Course combinations using CROSS JOIN.

```
31  SELECT COUNT(*) AS Possible_Combinations
32  FROM Student CROSS JOIN Course;
```

Result Grid

Possible_Combinations
60

15. Demonstrate a NATURAL JOIN between two suitable tables.

```
33  SELECT Student_ID, Name, Course_Name, Fees
34  FROM Student NATURAL JOIN Course;
```

Result Grid

Student_ID	Name	Course_Name	Fees
101	Anshika	B.Tech CSE	90000.00
102	Radha	B.Tech CSE	90000.00
103	Jayshree	B.Tech AIML	95000.00
105	Shreya	B.Tech CSE	90000.00
107	Rahul	B.Tech CSE	90000.00
108	Priya	BCA	60000.00
109	Aman	B.Tech IT	88000.00
110	Neha	B.Tech CSE	90000.00
111	Rohit	BCA	60000.00
112	Simran	B.Tech IT	88000.00

16. Compare the result of NATURAL JOIN and INNER JOIN.

```
35  SELECT s.Student_ID, s.Name, c.Course_Name, c.Fees
36  FROM Student s INNER JOIN Course c ON s.Course_ID = c.Course_ID AND s.Dept_ID = c.Dept_ID;
```

Result Grid

Student_ID	Name	Course_Name	Fees
101	Anshika	B.Tech CSE	90000.00
102	Radha	B.Tech CSE	90000.00
103	Jayshree	B.Tech AIML	95000.00
105	Shreya	B.Tech CSE	90000.00
107	Rahul	B.Tech CSE	90000.00
108	Priya	BCA	60000.00
109	Aman	B.Tech IT	88000.00
110	Neha	B.Tech CSE	90000.00
111	Rohit	BCA	60000.00
112	Simran	B.Tech IT	88000.00

17. Perform a SELF JOIN on an Employee table to show employee-manager relationships.

```
37  SELECT e.Emp_Name AS Employee, m.Emp_Name AS Manager
38  FROM Employee e LEFT JOIN Employee m ON e.Manager_ID = m.Emp_ID;
```

Result Grid

Employee	Manager
Aarav	NULL
Diya	Aarav
Kabir	Aarav
Meera	Aarav
Rohan	Aarav
Sneha	Kabir
Vikram	NULL
Pooja	Meera

18. Display employees along with their managers using SELF JOIN.

```
39  SELECT e.Emp_Name, m.Emp_Name AS Manager_Name
40  FROM Employee e INNER JOIN Employee m ON e.Manager_ID = m.Emp_ID;
```

Result Grid

Emp_Name	Manager_Name
Diya	Aarav
Kabir	Aarav
Meera	Aarav
Rohan	Aarav
Sneha	Kabir
Pooja	Meera

19. Join three tables—Student, Department and Course.

```
41  SELECT s.Name, d.Dept_Name, c.Course_Name
42  FROM Student s
43  JOIN Department d ON s.Dept_ID = d.Dept_ID
44  JOIN Course c ON s.Course_ID = c.Course_ID;
```

Result Grid

Name	Dept_Name	Course_Name
Anshika	Computer Science	B.Tech CSE
Radha	Computer Science	B.Tech CSE
Jayshree	Computer Science	B.Tech AIML
Shreya	Computer Science	B.Tech CSE
Rahul	Computer Science	B.Tech CSE
Priya	Computer Applications	BCA
Aman	Information Technology	B.Tech IT
Neha	Computer Science	B.Tech CSE
Rohit	Computer Applications	BCA
Simran	Information Technology	B.Tech IT

20. Display student name, course name and department name using multiple joins.

```
45  SELECT s.Name, c.Course_Name, d.Dept_Name
46  FROM Student s
47  JOIN Course c ON s.Course_ID = c.Course_ID
48  JOIN Department d ON c.Dept_ID = d.Dept_ID;
```

Result Grid

Name	Course_Name	Dept_Name
Anshika	B.Tech CSE	Computer Science
Radha	B.Tech CSE	Computer Science
Jayshree	B.Tech AIML	Computer Science
Ruhani	B.Tech CSE	Computer Science
Shreya	B.Tech CSE	Computer Science
Maya	B.Tech CSE	Computer Science
Rahul	B.Tech CSE	Computer Science
Priya	BCA	Computer Applications
Aman	B.Tech IT	Information Technology
Neha	B.Tech CSE	Computer Science
Rohit	BCA	Computer Applications
Simran	B.Tech IT	Information Technology

21. Find employees whose salary is greater than the average salary of their department using a join and subquery.

```
49  SELECT e.Emp_Name, e.Salary, d.Dept_Name
50  FROM Employee e JOIN Department d ON e.Dept_ID = d.Dept_ID
51  WHERE e.Salary > (SELECT AVG(e2.Salary) FROM Employee e2 WHERE e2.Dept_ID = e.Dept_ID);
```

Result Grid

Emp_Name	Salary	Dept_Name
Sneha	58250.25	Information Technology
Vikram	82000.00	Computer Science

22. Find the department having the maximum number of students using joins and aggregation.

```
52  SELECT d.Dept_Name, COUNT(s.Student_ID) AS Student_Count
53  FROM Department d LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
54  GROUP BY d.Dept_ID, d.Dept_Name
55  ORDER BY Student_Count DESC LIMIT 1;
```

Result Grid

Dept_Name	Student_Count
Computer Science	6

23. Display departments and the total number of students in each department.

```
56  SELECT d.Dept_Name, COUNT(s.Student_ID) AS Student_Count
57  FROM Department d LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
58  GROUP BY d.Dept_ID, d.Dept_Name;
```

Result Grid

Dept_Name	Student_Count
Computer Science	6
Information Technology	2
Computer Applications	2
Mechanical	0
Civil	0
Human Resources	0

24. Display courses along with the number of students enrolled in each course.

```
59  SELECT c.Course_Name, COUNT(s.Student_ID) AS Enrolled_Students
60  FROM Course c LEFT JOIN Student s ON s.Course_ID = c.Course_ID
61  GROUP BY c.Course_ID, c.Course_Name;
```

Result Grid

Course_Name	Enrolled_Students
B.Tech CSE	7
B.Tech AIML	1
B.Tech IT	2
BCA	2
B.Tech Civil	0

25. Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions.

```
62  SELECT d.Dept_Name, c.Course_Name, COUNT(s.Student_ID) AS Students, AVG(s.Age) AS Average_Age
63  FROM Department d
64  INNER JOIN Course c ON c.Dept_ID = d.Dept_ID
65  LEFT JOIN Student s ON s.Course_ID = c.Course_ID
66  GROUP BY d.Dept_ID, d.Dept_Name, c.Course_ID, c.Course_Name;
```

Result Grid

Dept_Name	Course_Name	Students	Average_Age
Computer Science	B.Tech CSE	7	20.1429
Computer Science	B.Tech AIML	1	17.0000
Information Technology	B.Tech IT	2	22.0000
Computer Applications	BCA	2	20.0000
Civil	B.Tech Civil	0	NULL